6.101 Final

answers

Fall 2025

- You have 3 hours to complete this exam. There are **8 problems**.
- **This exam is closed-book**.
    - You may use blank scratch paper.
    - You may **not** use any notes or cheat sheets. You may **not** use other electronics (including calculators, phones, tablets, music players, *etc.*) and you may **not** use other web sites or programs (including messaging platforms, search engines, LLMs, IDEs, documentation, editors, the Python interpreter, the course website, *etc.*).
    - Before the exam starts, **close all other applications, windows, and browser tabs on your laptop**, so that you can take the exam without any unintended interruptions. You should also **disable notifications** on your devices.
- Before you begin: you must **check in** by having the course staff scan the QR code at the top of the page.
- This page **automatically saves your answers** as you work. If you see a stuck yellow spinner, red exclamation mark, or a red notification that you are disconnected, your answers are not being saved: try reloading the page right away, before continuing to work on the exam.
- If you feel the need to **write a note to the grader**, you can click the gray pencil icon to the right of the answer.
- If you have a question, or need to use the restroom, please raise your hand.
- If you find yourself bogged down on one part of the exam, remember to keep going and work on other problems, and then come back.
- To **leave early**: enter *done* at the very bottom of the page and show your screen with the check-out code to a staff member.
- You **may not discuss** details of the exam with anyone other than course staff until grades have been released.

Good luck!

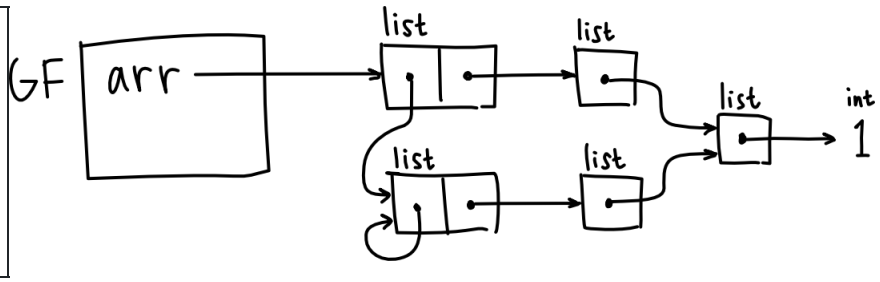
Problem 1

For each environment diagram below, write code that results in the diagram. Do **not** bind any additional variables or attributes, even if you subsequently delete them. Do **not** create any additional frames or objects, even if they could be garbage collected after running your code, except that you may use extra `int` values like `0`, and literal `None`.

Parts of each diagram are tricky! If you get stuck, come back to them later.

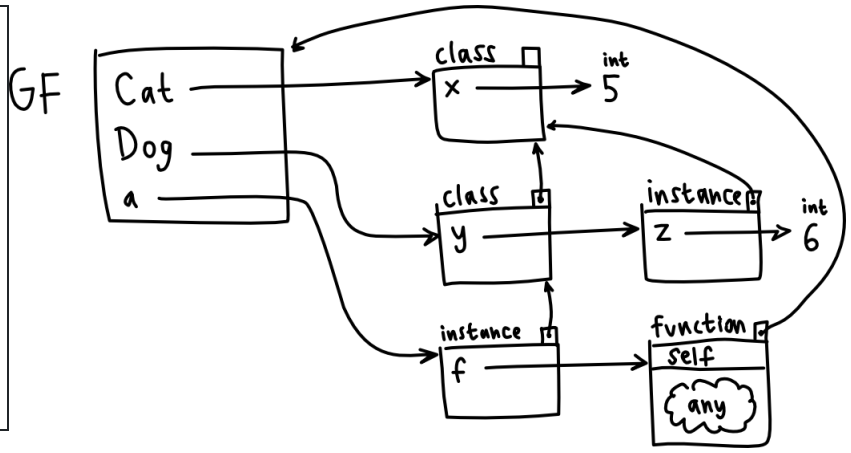
Lists?

```
arr = [[None, [None]], [[1]]]
arr[0][0] = arr[0]
arr[0][1] = [arr[1][0]]
```



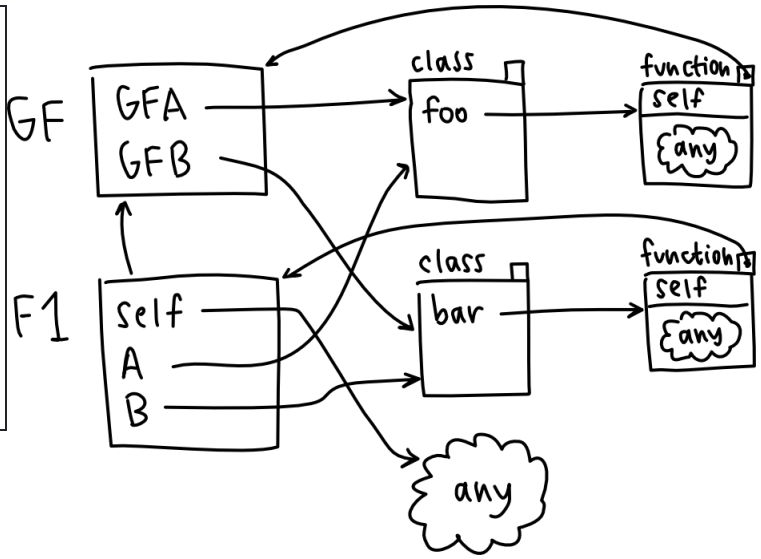
Cats and Dogs! The body of the function object can be any code you choose.

```
class Cat:
    x = 5
class Dog(Cat):
    y = Cat()
    y.z = 6
a = Dog()
a.f = lambda self: None
```



Uh... **gfagfb!?** The bodies of the two function objects can be any code you choose, and the object pointed to by `self` in F1 can be any object.

```
class GFA:
    def foo(self):
        A = GFA
        class B:
            def bar(self):
                pass
        return B
GFB = GFA().foo()
```



Reminder: do not bind any additional names or create any additional objects not shown in the diagrams.

Problem 2

Coming up in Problem 3, we will use the built-in function **`sorted`** on collections of tuples. To make comparisons between tuples, Python uses **lexicographical ordering**. The documentation explains that to compare two tuples...

> ... the first items in each tuple are compared, and if they differ this determines the outcome of the comparison; if they are equal, the next two items are compared, and so on, until either tuple is exhausted. If all items of two tuples compare equal, the tuples are considered equal. If one tuple is an initial sub-sequence of the other, the shorter tuple is less than the longer one.

Implement this behavior by writing a **recursive** function, without using any built-in comparison operations on tuples.

Your implementation may **not** iterate over an input or access elements at arbitrarily-large indices.

```
def less_than(tup1, tup2):
    """
    Given two tuples of zero or more numbers, returns True if and only if
    tup1 is lexicographically less than tup2, and returns False otherwise.
    >>> less_than((1, 2, 3), (1, 2, 4))
    True
    >>> less_than((1, 2, 3, 4), (1, 2, 4))
    True
    >>> less_than((1, 2), (1, 2, 4))
    True
    >>> less_than((1, 2), (1, 2))
    False
    """
```

```
if not tup2:
    return False
if not tup1 or tup1[0] < tup2[0]:
    return True
if tup1[0] > tup2[0]:
    return False
return less_than(tup1[1:], tup2[1:])
```

Problem 3

On the midterm, we wrote code to solve Pips puzzles. You can find the complete description of Pips at the end of the exam, where you can also open it in a new tab. As a quick reminder:

> **Domino pieces** are rectangular game pieces divided into two squares, with zero to six dots (technical term: *pips*) on each square.
>
> We represent dominoes as 2-element tuples of integers `0` through `6`.
>
> A **Pips puzzle** consists of a set of different domino pieces and an arbitrarily-shaped board. Regions of the board are annotated with conditions, like "the sum of dots in this region must equal a number" or "all squares in this region must have the same number of dots." Some squares on the board might have no condition. The goal is to place all the dominoes to cover the entire board, satisfying the conditions.
>
> We represent Pips puzzles using dictionaries with three keys:
>
> ```
> puzzle = {
>     "constraints": ...a dictionary of string region names to constraint functions...
>     "board": ...a dictonary of (row, col) tuples to region names or None...,
>     "dominoes": ...a set of domino tuples...,
> }
> ```
>
> Remember that the constraint functions return `True` if and only if the constraint is **or could be** satisfied, a change we made during the midterm as we wrote our solver.
>
> And we represent a **solution** with a dictionary where the keys are non-rotated dominoes and the values are tuples of *(row, column, rotation)* tuples, where *rotation* is the number of 90° clockwise turns: `0`, `1`, `2`, or `3`.
>
> Now is a good time to review the examples in the full description at the end of the exam and open that description in a new tab.

The midterm asked you to implement several helper functions, then debug a broken implementation of `solve(..)` that used our search function `find_path(..)` from the reading.

You can find the documentation and code for the three helper functions **at the end of the exam**. They are:

- `expand_solution(solution)`
- `solution_covers_board(puzzle, solution)`
- `solution_satisfies_constraints(puzzle, solution)`

Consider this version of `find_path`, whose only change from the reading is some debugging code: two calls to `print(..)` that use a new parameter `state_debug`:

```
def find_path(neighbors_function, start, goal_test, state_debug): # ← new parameter state_debug
    if goal_test(start):
        return (start, )
    agenda = [(start, )]
    visited = {start}
    while agenda:
        this_path = agenda.pop(0)
        terminal_state = this_path[-1]
        print("path ending", state_debug(terminal_state)) # ← debugging print
        for neighbor in neighbors_function(terminal_state):
            if neighbor not in visited:
                print("neighbor", state_debug(neighbor)) # ← debugging print
```

```
                new_path = this_path + (neighbor,)
                if goal_test(neighbor):
                    return new_path
                agenda.append(new_path)
                visited.add(neighbor)
    return None
```

And suppose we write this implementation of Pips puzzle solving, where `neighbors` is a generator function:

```
def solve_with_find_path(puzzle):
    """
    Returns a solution to the puzzle, or None if it cannot be solved.
    """

    def solution_to_state(solution):
        """Represent a solution as a tuple of sorted (key, value) dict item tuples."""
        return tuple(sorted(solution.items()))

    def state_to_solution(state):
        """Re-create a solution dict from its tuple representation."""
        return {domino: placement for (domino, placement) in state}

    def neighbors(state):
        solution = state_to_solution(state)
        # for each domino in lexicographical order
        for domino in sorted(puzzle["dominoes"]):
            # try to place it at coordinates in lexicographical order
            for (row, col) in sorted(puzzle["board"]):
                for rotate in range(4):
                    neighbor = solution | {domino: (row, col, rotate)}
                    if solution_satisfies_constraints(puzzle, neighbor):
                        yield solution_to_state(neighbor)

    def goal(state):
        solution = state_to_solution(state)
        return (solution_covers_board(puzzle, solution) and
                solution_satisfies_constraints(puzzle, solution))

    start = solution_to_state({})
    path = find_path(neighbors, start, goal, state_to_solution) # ← now takes 4 arguments
    return state_to_solution(path[-1]) if path else None
```

This implementation uses the hashable state representation we saw on the midterm: a solution dictionary like

`{(6, 6): (0, 1, 2), (1, 2): (1, 0, 0)}`

is converted to and from a lexicographically-sorted tuple of key-value pair tuples:

`(((1, 2), (1, 0, 0)), ((6, 6), (0, 1, 2)))`

And the line

`neighbor = solution | {domino: (row, col, rot)}`

uses a dictionary union to create a new dictionary that combines `solution` on the left with the single-key dictionary right. For example:

```
>>> {"a": 1} | {"b": 2}
{"a": 1, "b": 2}
```

These implementations are reproduced **at the end of the exam** so you can open them in a separate tab.

What will be the output if we run the following code in the Python REPL?

```
>>> trivial = {"constraints": {},
...            "board": {(0, 0): None, (1, 0): None},
...            "dominoes": {(6, 6)} }
>>> solve_with_find_path(trivial)
```

The first line of output is provided, you should write the remaining lines:

```
path ending {}
```

```
neighbor {(6, 6): (0, 0, 1)}
{(6, 6): (0, 0, 1)}
```

And what will be the output if we run the following code?

```
>>> def total_is(total):
...     return lambda nums: (sum(0 if n is None else n for n in nums) <= total
...                          <= sum(6 if n is None else n for n in nums))
>>> impossible = {"constraints": {"six": total_is(6), "one": total_is(1)},
...               "board": {(0, 0): "six", (0, 1): "six", (0, 2): "one", (0, 3): None},
...               "dominoes": {(1, 6), (2, 3)}}
>>> solve_with_find_path(impossible)
```

The first line of output is provided, you should write the remaining lines:

```
path ending {}
```

```
neighbor {(1, 6): (0, 2, 0)}
neighbor {(1, 6): (0, 2, 2)}
path ending {(1, 6): (0, 2, 0)}
path ending {(1, 6): (0, 2, 2)}
```

Problem 4

Let's focus just on the debugging code in the previous problem:

```
def find_path(neighbors_function, start, goal_test, state_debug):
    ...
    print("path ending", state_debug(terminal_state))
    ...
        print("neighbor", state_debug(neighbor))
    ...

def solve_with_find_path(puzzle):
    ...
    def state_to_solution(state):
        return {domino: placement for (domino, placement) in state}
    ...
    path = find_path(neighbors, start, goal, state_to_solution)
    ...
```

For each of the following alternative ways to write these lines, state whether or not the new way works as desired, and explain clearly why or why not in at most two sentences.

Alternative #1:

```
def find_path(neighbors_function, start, goal_test): # ← changed here
    ...
    print("path ending", state_debug(terminal_state))
    ...
        print("neighbor", state_debug(neighbor))
    ...

def solve_with_find_path(puzzle):
    ...
    def state_to_solution(state):
        return {domino: placement for (domino, placement) in state}
    def state_debug(state):                  # ← added line
        return state_to_solution(state)      # ← added line
    ...
    path = find_path(neighbors, start, goal) # ← changed here
    ...
```

Does this work? Why or why not?

No. In find_path, state_debug is not defined, so that function exits with a NameError.

Alternative #2:

```
def find_path(neighbors_function, start, goal_test, state_debug):
    ...
    print("path ending", state_debug(terminal_state))
    ...
        print("neighbor", state_debug(neighbor))
    ...

def solve_with_find_path(puzzle):
    ...
    def state_to_solution(state):
        return {domino: placement for (domino, placement) in state}
    state_debug = state_to_solution(state)  # ← added line
    ...
    path = find_path(neighbors, start, goal, state_debug) # ← changed here
    ...
```

Does this work? Why or why not?

No. On the added line, state is not defined, and solve_with_find_path will exit with a NameError.

Alternative #3:

```
def find_path(neighbors_function, start, goal_test, state_debug):
    ...
    print("path ending", state_debug(terminal_state))
    ...
        print("neighbor", state_debug(neighbor))
    ...

def solve_with_find_path(puzzle):
    ...
    def state_to_solution(state):
        return {domino: placement for (domino, placement) in state}
    def state_debug(state):             # ← added line
        return state_to_solution        # ← added line
    ...
    path = find_path(neighbors, start, goal, state_debug) # ← changed here
    ...
```

Does this work? Why or why not?

No. Instead of returning a solution dictionary, calling state_debug returns the function state_to_solution, whose string representation will be printed.

Alternative #4:

```
def find_path(neighbors_function, start, goal_test): # ← changed here
    ...
    print("path ending", state_debug(terminal_state))
    ...
        print("neighbor", state_debug(neighbor))
    ...

def solve_with_find_path(puzzle):
    ...
    def state_to_solution(state):
        return {domino: placement for (domino, placement) in state}
    ...
    path = find_path(neighbors, start, goal) # ← changed here
    ...

def state_debug(state):                      # ← added line
    return state_to_solution(state)          # ← added line
```

Does this work? Why or why not?

No. In state_debug, state_to_solution is not defined, so that function exits with a NameError.

Problem 5

Now implement Pips puzzle solving using **recursive backtracking** instead. You may use the same three helper functions:

- `expand_solution(solution)`
- `solution_covers_board(puzzle, solution)`
- `solution_satisfies_constraints(puzzle, solution)`

Do **not** use `find_path`, and do **not** use other helpers that are not in scope. Your solution must be recursive.

You may not change the parameters of `solve`, so consider defining and calling a recursive helper function in its body.

For full credit, your solution should be straightforward, readable, and efficient, without using "agenda" or "visited" data structures.

```
def solve(puzzle):
    """
    Returns a solution to the puzzle, or None if it cannot be solved. Does not modify puzzle.
    """
```

```
    def helper(sofar):
        if not solution_satisfies_constraints(puzzle, sofar):
            return None
        if solution_covers_board(puzzle, sofar):
            return sofar
        domino = (puzzle["dominoes"] - sofar.keys()).pop()
        for row, col in puzzle["board"].keys():
            for rot in range(4):
                result = helper(sofar | {domino: (row, col, rot)})
                if result is not None:
                    return result
        return None

    return helper({})
```

Briefly explain why your solution is efficient: what unnecessary or duplicate work does it avoid, and how does it avoid that work? Do not write more than two sentences.

It does not explore adding to a partial solution if it already violates the constraints, and only explores placing a single next domino since all dominoes must be placed.

Problem 6

Pips puzzles are not required to have a unique solution.

- Symmetrical dominoes, for example, can always be rotated 180°.
- Depending on the dominoes available and the shape of the board, there might be multiple different placements of the dominoes that put the same number of pips in each square.
- And depending on the constraints, there might be multiple solutions with entirely different arrangements of dominoes!

So instead of returning *one* solution, write a **recursive backtracking generator function** that will produce *all* valid solutions to a puzzle. The generator should include each distinct puzzle solution *once*. Your implementation may **not** build any list(s) or other collection(s) of partial or complete solutions.

You may not change the parameters of `solutions`, so consider defining and calling a recursive helper function and using `yield from`.

Before solving this problem, first revise your recursive backtracking `solve(..)` to your satisfaction, so you do not have duplicate bugs or need to make duplicate fixes!

For full credit, your solution should be straightforward, readable, and efficient.

```
def solutions(puzzle):
    """
    A generator of all distinct valid solutions to the puzzle. Does not modify puzzle.
    """
```

```
def helper(sofar):
    if not solution_satisfies_constraints(puzzle, sofar):
        return
    if solution_covers_board(puzzle, sofar):
        yield sofar
        return
    domino = (puzzle["dominoes"] - sofar.keys()).pop()
    for row, col in puzzle["board"].keys():
        for rot in range(4):
            yield from helper(sofar | {domino: (row, col, rot)})

yield from helper({})
```

Problem 7

Another way to solve Pips puzzles would be to encode them as instances of the Boolean satisfiability problem, and solve them with a SAT solver.

In the SAT solving lab, we represented formulas in conjunctive normal form (CNF) in the following way:

- a *variable* as a Python string
- a *literal* as a pair (a tuple), containing a variable and a Boolean value (`False` if `not` appears in the literal, `True` otherwise)
- a *clause* as a list of literals
- a *formula* as a list of clauses

What if we instead created classes to represent CNF formulas, clauses, and literals?

Given the following formula:

```
two_flavors = ((not chocolate or not vanilla or not pickles)
               and (chocolate or vanilla) and (vanilla or pickles) and (pickles or chocolate))
```

... (which encodes a requirement that exactly two out of `chocolate`, `vanilla`, and `pickles` must be `True`), we can represent it as:

```
pos_choco = Literal("chocolate") # a "positive" literal is a variable, here: chocolate
neg_choco = pos_choco.no()       # a "negative" literal is a negated variable, here: not chocolate
not_all_three = Clause([neg_choco, Literal("vanilla").no(), Literal("pickles").no()])
two_flavors = Formula([not_all_three,
                       Clause([pos_choco, Literal("vanilla")]),
                       Clause([Literal("vanilla"), Literal("pickles")]),
                       Clause([Literal("pickles"), pos_choco])])
```

We might write our SAT solver as follows:

```
def satisfying_assignment(formula):
    """
    Find a satisfying assignment for a given CNF Formula.
    Returns an assignment if one exists, or None otherwise, and does not mutate formula.
    The assignment may omit variables that are unconstrained.
    >>> satisfying_assignment(Formula([Clause([pos_choco])]))
    {'chocolate': True}
    >>> assgn = satisfying_assignment(two_flavors) # several correct answers...
    >>> sorted(assgn.keys()), sorted(assgn.values()) # ... so just check keys and values
    (['chocolate', 'pickles', 'vanilla'], [False, True, True])
    """
    if len(formula) == 0:
        return TODO
    for literal in formula.small_clause():
        result = satisfying_assignment(formula.assuming(literal.variable, literal.positive))
        if TODO:
            return result | {literal.variable: literal.positive}
    return TODO
```

What is the correct return value on line 2?

```
{}
```

... and in a single clear sentence: why?

An empty formula is trivially satisfied without making any assignments.

What is the correct condition on line 5?

```
result is not None
```

... and in a single clear sentence: why?

A non-None (but potentially empty) result indicates a satisfying assignment for the smaller formula which we can build on to satisfy the larger formula.

What is the correct return value on line 7?

```
None
```

... and in a single clear sentence: why?

If none of the literals in the smallest clause can be satisfied, that clause cannot be satisfied, the formula cannot be satisfied, and we return None.

Implement `Formula`, `Clause`, and `Literal` so that they work with the code above. The requirements are reiterated below. You are free to include additional functionality consistent with the requirements.

Do **not** write docstrings for the functionality we require. In all other respects your code should be exemplary to receive full credit.

```
class Formula:
    """
    A CNF formula: a conjunction (AND) of disjunctions (ORs) of literals (variables and negated variab
    Provides an initializer that takes a list of Clause objects.
    Instances provide:
     - __len__, so that len(formula) returns the number of clauses in formula
     - small_clause, which returns some smallest clause in the formula, or None if there are none
     - assuming, which takes a variable and True or False, and returns a new Formula assuming the
           variable has that value (clauses that are satisfied and literals that cannot be true are
           removed)
     - (other operations that you decide, but none may mutate the Formula)
    """
```

```
def __init__(self, clauses):
    self.clauses = clauses

def __len__(self):
    return len(self.clauses)

def small_clause(self):
    return min(self.clauses, key=len, default=None)
```

```
def assuming(self, variable, positive):
    return Formula([
        clause.without(variable)
        for clause in self.clauses
        if not clause.satisfied_by(variable, positive)
    ])
```

class Clause:

```
"""
A clause in a CNF formula: a disjunction (OR) of literals (variables and negated variables).
Provides an initializer that takes a list of Literal objects.
Instances provide:
 - __iter__, so that iterating over a Clause is iteration over its Literals (can return
     any iterable or generator of Literals)
 - (other operations that you decide, but none may mutate the Clause)
"""
```

```
def __init__(self, literals):
    self.literals = literals

def __len__(self):
    return len(self.literals)

def __iter__(self):
    yield from self.literals

def satisfied_by(self, variable, positive):
    """Returns True iff the given assumption satisfies the clause."""
    return any(lit.matches(variable, positive) for lit in self.literals)

def without(self, variable):
    """Returns the clause, omitting literals containing the given variable."""
    return Clause([
        literal
        for literal in self.literals
        if literal.variable != variable
    ])
```

```
class Literal:
    """
    A literal in a formula: a variable (called a positive literal) or negated variable (negative).
    Provides an initializer that takes a string variable name and makes its positive literal.
    Instances provide:
     - no, which returns the negation of the literal (positive to negative, negative to positive)
           as a new Literal
     - variable (attribute), the string variable name
     - positive (attribute), True if the literal is positive (just the variable), False otherwise
     - (other operations that you decide, but none may mutate the Literal)
    """
```

```
def __init__(self, variable, positive=True):
    self.variable = variable
    self.positive = positive

def no(self):
    return Literal(self.variable, not self.positive)

def matches(self, variable, positive):
    """Returns True iff the literal has the given variable and positivity."""
    return self.variable == variable and self.positive == positive
```

Problem 8

This semester we only worked with Boolean formulas in conjunctive normal form. But in general, a Boolean formula might nest conjunctions (AND) and disjunctions (OR) arbitrarily deeply.

Here are some examples of a textual language for Boolean formulas, along with their translation into an equivalent Python expression:

Boolean formula language	Python equivalent
`"choco"`	`choco`
`"AND(choco -vanil pickle)"`	`choco and not vanil and pickle`
`"OR(AND(a b) -c)"`	`(a and b) or not c`
`"OR(a AND(b c) d)"`	`a or (b and c) or d`

In our language:

- variables are lower-case alphabetic letter strings
- negation is indicated with a minus sign: only variables can be negated (not larger sub-formulas, and not negations themselves)
- conjunction and disjunction are indicated with `AND` and `OR` followed by one or more sub-formulas within parentheses

Suppose we have already implemented tokenization:

```
def tokenize(source):
    """
    Returns the list of individual token strings in the source string.
    >>> tokenize("choco")
    ['choco']
    >>> tokenize("AND(choco -vanil pickle)")
    ['AND', '(', 'choco', '-', 'vanil', 'pickle', ')']
    >>> tokenize("OR(AND(a b) -c)")
    ['OR', '(', 'AND', '(', 'a', 'b', ')', '-', 'c', ')']
    >>> tokenize("OR(a AND(b c) d)")
    ['OR', '(', 'a', 'AND', '(', 'b', 'c', ')', 'd', ')']
    >>> tokenize("BROKEN()AND)OR(")  # so many issues, but it tokenizes
    ['BROKEN', '(', ')', 'AND', ')', 'OR', '(']
    """
    …omitted…
```

Implement parsing for this language. The output of `parse` should be a tree-like representation of the formula, where:

- a literal is represented with an instance of the `Literal` class above
- conjunctions and disjunctions are S-expression-like lists, where the first element is either the string `"AND"` or `"OR"` and subsequent elements are the parsed sub-formulas.

Your implementation should try to identify invalid inputs. Instead of returning a broken parse tree or causing an implementation error, like `IndexError` or `TypeError`, use **`assert`** to identify specific error conditions and cause an `AssertionError`. You do **not** need to construct error messages for the assertions.

Your implementation should rely *only* on the required functionality of `Literal`, **not** on any additional methods or attributes you may have added.

Consider defining and using a recursive parsing helper function that returns a next index in addition to the parsed subexpression.

```
def is_varname(s):
    """
    Return True if and only if the input string is nonempty, alphabetic, and lower case.
    """
    return s.isalpha() and s.islower()

def parse(tokens):
    """
    Parses the tokens list into a tree-like representation as described above.
    >>> parse(tokenize("choco"))
    Literal('choco')
    >>> parse(tokenize("AND(choco -vanil pickle)"))
    ['AND', Literal('choco'), Literal('vanil').no(), Literal('pickle')]
    >>> parse(tokenize("OR(AND(a b) -c)"))
    ['OR', ['AND', Literal('a'), Literal('b')], Literal('c').no()]
    >>> parse(tokenize("OR(a AND(b c) d)"))
    ['OR', Literal('a'), ['AND', Literal('b'), Literal('c')], Literal('d')]
    """
```

```
def parse_at(index):
    assert index < len(tokens)
    if tokens[index] in {"AND", "OR"}:
        assert index+1 < len(tokens) and tokens[index+1] == "("
        result = [tokens[index]]
        last = index+2
        assert last < len(tokens) and tokens[last] != ")"
        while tokens[last] != ")":
            formula, last = parse_at(last)
            result.append(formula)
            assert last < len(tokens)
        return result, last + 1
    if tokens[index] == "-":
        literal, last = parse_at(index+1)
        assert isinstance(literal, Literal)
        return literal.no(), last
    assert is_varname(tokens[index])
    return Literal(tokens[index]), index+1

expr, last = parse_at(0)
assert last == len(tokens)
return expr
```

Those doctests for `parse(..)` will only pass if we implement dunder method **`__repr__`** for `Literal` instances in the way they expect. Do that now, again relying *only* on the required functionality of `Literal`, not on any additional methods or attributes you may have added.

(Remember that `obj.__repr__()` is intended to return a string that is a Python expression we could use to recreate `obj`.)

`class Literal`:

…
… *some implementation that provides the required functionality* …
…

```
def __repr__(self):
    return f"Literal('{self.variable}'){'' if self.positive else '.no()'}"
```

Pips

You can **open this entire section in a new tab** so that you can refer to it more easily.

The *New York Times* recently introduced a new game called Pips. (Pips is edited by Ian Livengood. All images below © 2025 The New York Times Company.)

A **Pips puzzle** consists of a set of different domino pieces and an arbitrarily-shaped board.

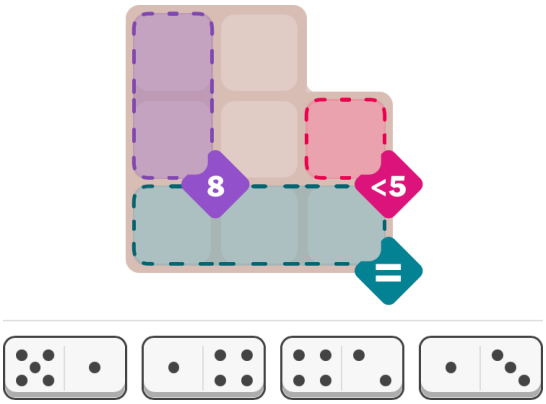
Regions of the board are annotated with condititons, for example:

- the sum of dots in that region must equal a certain number
- the sum of dots must be less than a certain number; or greater than a certain number
- all of the squares in the region must have the same number of dots; or must have different numbers of dots

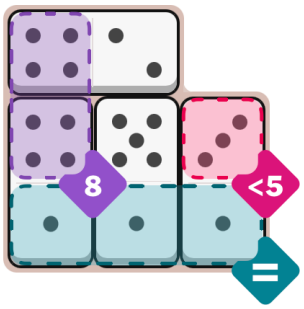
Some squares on the board might have no condition.

The goal is to place all the dominoes to cover the entire board, satisfying the conditions.

For example, here is an “easy” puzzle:



And here is its solution:



We will represent a Pips puzzle using a dictionary with three keys:

```
puzzle = {
    "constraints": ...a dictonary...,
    "board": ...a dictonary...,
    "dominoes": ...a set...,
}
```

1. The `"constraints"` dictionary is a dictionary of strings to functions.

   The string keys are arbitrary, unique names for each region of the board that has a constraint.

   Each value, a function, takes as input a list of the number of dots in every square of the region: integer elements `0` through `6` indicate that number of dots, or `None` indicates no domino. The function returns `True` if the constraint on the region is or could be satisfied, or `False` if the constraint is not or cannot be satisfied.

   For our “easy” example, we could write:

```
"constraints": {
    "makeEight": lambda nums: (sum(0 if n is None else n for n in nums) <= 8
                               <= sum(6 if n is None else n for n in nums))
    "lessThanFive": lambda nums: nums[0] is None or nums[0] < 5,
    "all3Equal": lambda nums: len(set(nums) - {None}) <= 1,
}
```

2. The `"board"` dictionary is a dictionary of 2-element tuples to strings or `None`.

   The keys of the dictionary are (row, column) coordinates, counting from the top left of the puzzle, down and to the right. The board shape is not constrained: coordinates with no board square are *omitted* from the dictionary.

   Each value in the dictionary is a string, indicating that square belongs to the constrained region with that unique name in the `"constraints"` dictionary; or `None`, indicating an unconstrained square.

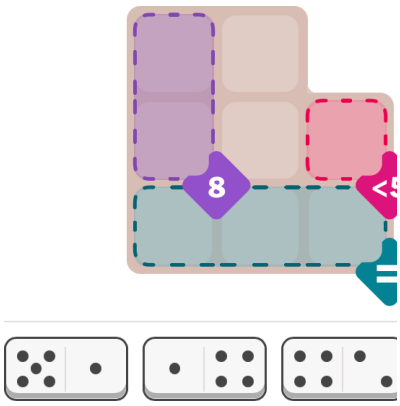
   ```
   "board": {
       (0, 0): "makeEight", (0, 1): None,         # no square (0, 2) on this board
       (1, 0): "makeEight", (1, 1): None,         (1, 2): "lessThanFive",
       (2, 0): "all3Equal", (2, 1): "all3Equal", (2, 2): "all3Equal",
   }
   ```

3. And the `"dominoes"` set is a set of 2-element tuples of integers 0 through 6 representing the available dominoes in their initial non-rotated orientation:

   ```
   "dominoes": {(5, 1), (1, 4), (4, 2), (1, 3)}
   ```

All together, here is our initial representation for the “easy” example:

```
easy = {
    "constraints": {
        "makeEight": lambda nums: …see above…,
        "lessThanFive": lambda nums: …see above…,
        "all3Equal": lambda nums: …see above…
    },
    "board": {
        (0, 0): "makeEight", (0, 1): None,
        (1, 0): "makeEight", (1, 1): None, (1, 2): "lessThanFive",
        (2, 0): "all3Equal", (2, 1): "all3Equal", (2, 2): "all3Equal"
    },
    "dominoes": {(5, 1), (1, 4), (4, 2), (1, 3)}
}
```



To represent a **solution** to a puzzle, we use a dictionary where the non-rotated dominoes are the keys, and the values are 3-element tuples of integers:

- first the row and column position of the first listed square of the domino
- then a rotation value of `0`, `1`, `2`, or `3` that specifies where the second square of the domino is, relative to the first:

  `0` = to the right (domino is not rotated) *e.g.* 

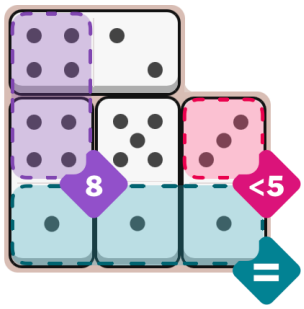
  `1` = below (domino is rotated 90° clockwise around the first listed square) 

  `2` = to the left (domino is rotated 180°) 

  `3` = above (domino is rotated 270°) 

For example:

```
easy_solution = {
    (5, 1): (1, 1, 1),
    (1, 4): (2, 0, 3),
    (4, 2): (0, 0, 0),
    (1, 3): (2, 2, 3),
}
```

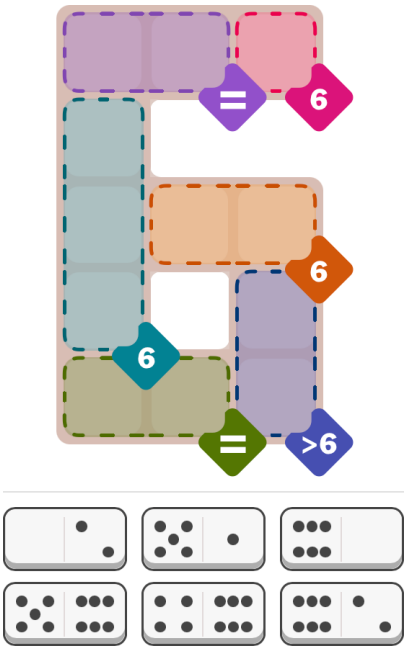


We will use the same format to represent partial solutions, where only some dominoes are placed:

```
easy_in_progress = {
    (5, 1): (1, 1, 1),
    (1, 4): (2, 0, 3),
}
```

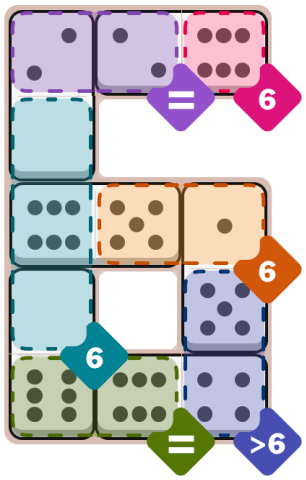
Finally, here is an example of a more complicated puzzle. You don't need to read it in detail, but notice how multiple regions can have identical constraints. Our representation gives different names (*e.g.* `"purple"` and `"green"`) to each region, even if their constraints are the same. The *Times* uses colors to help distinguish regions, and we have used those same colors as unique identifiers, but we could use any unique strings.

```
hard = {
    "constraints": {
        "purple": lambda nums: …omitted…,
        "pink": lambda nums: …omitted…,
        "aqua": lambda nums: …omitted…,
        "orange": lambda nums: …omitted…,
        "green": lambda nums: …omitted…,
        "blue": lambda nums: …omitted…,
    },
    "board": {
        (0, 0): "purple", (0, 1): "purple", (0, 2): "pink",
        (1, 0): "aqua",
        (2, 0): "aqua",   (2, 1): "orange", (2, 2): "orange",
        (3, 0): "aqua",                     (3, 2): "blue",
        (4, 0): "green",  (4, 1): "green",  (4, 2): "blue",
    },
    "dominoes": {(0, 2), (5, 1), (0, 6), (5, 6), (4, 6), (6, 2)}
}
```



Here is the solution to the “hard” puzzle:

```
hard_solution = {
    (0, 2): (1, 0, 3),
    (5, 1): (3, 2, 3),
    (0, 6): (3, 0, 1),
    (5, 6): (2, 1, 2),
    (4, 6): (4, 2, 2),
    (6, 2): (0, 2, 2)
}
```



During the midterm we implemented **three solving helper functions**:

```
def expand_solution(solution):
    """
    Given a solution dictionary (mapping dominoes to their locations & orientations),
    returns a dictionary whose keys are all the (row, column) board squares covered
    and the values are the number of dots in those squares. If the solution specifies
    overlapping dominoes or squares at negative coordinates, returns None.
    >>> expand_solution(easy_solution) == {
    ...     (0, 0): 4,  (0, 1): 2,
    ...     (1, 0): 4,  (1, 1): 5,  (1, 2): 3,
    ...     (2, 0): 1,  (2, 1): 1,  (2, 2): 1}
    True
    >>> expand_solution({}) == {}
    True
    >>> expand_solution({(5, 1): (1, 1, 1)}) == {
    ...     (1, 1): 5,
    ...     (2, 1): 1}
    True
    >>> expand_solution({(5, 1): (1, 1, 1),
    ...                  (1, 4): (2, 0, 0)}) is None
    True
    """
    result = {}
    rotations = [(0, 1), (1, 0), (0, -1), (-1, 0)]
    for left, right in solution:
        row, col, rot = solution[left, right]
        dr, dc = rotations[rot]
        if (row, col) in result:
            return None
        if (row+dr, col+dc) in result:
            return None
        if row < 0 or col < 0 or row+dr < 0 or col+dc < 0:
            return None
        result |= {(row, col): left, (row+dr, col+dc): right}
    return result
```

```
def solution_covers_board(puzzle, solution):
    """
    Test whether solution exactly covers puzzle["board"] using puzzle["dominoes"].
    """
    if puzzle["dominoes"] != solution.keys():
        return False
    expanded = expand_solution(solution)
    if expanded is None:
        return False
    return puzzle["board"].keys() == expanded.keys()


def solution_satisfies_constraints(puzzle, solution):
    """
    Given a puzzle dictionary and a solution dictionary, returns True if solution is
    consistent with the puzzle board (e.g. no dominoes placed outside the board) and
    constraints (all are satisfied or could be satisfied). Returns False otherwise.
    >>> solution_satisfies_constraints(easy, easy_solution)    # complete solution
    True
    >>> solution_satisfies_constraints(easy, easy_in_progress) # partial solution
    True
    >>> solution_satisfies_constraints(easy, {})               # no progress
    True
    >>> solution_satisfies_constraints(easy, {(5, 1): (1, 2, 1)}) # violates lessThanFive
    False
    >>> solution_satisfies_constraints(easy, {(1, 4): (2, 0, 1)}) # outside the board
    False
    """
    expanded = expand_solution(solution)
    if expanded is None:
        return False
    if expanded.keys() - puzzle["board"].keys():
        return False
    return all(
        predicate([expanded.get(coord)
                   for coord in puzzle["board"]
                   if puzzle["board"][coord] == region])
        for region, predicate in puzzle["constraints"].items()
    )
```

These implementations are for **problem 3 only**:

```
    def find_path(neighbors_function, start, goal_test, state_debug): # ← new parameter state_debug
        if goal_test(start):
            return (start, )
        agenda = [(start, )]
        visited = {start}
        while agenda:
            this_path = agenda.pop(0)
            terminal_state = this_path[-1]
            print("path ending", state_debug(terminal_state)) # ← debugging print
            for neighbor in neighbors_function(terminal_state):
                if neighbor not in visited:
                    print("neighbor", state_debug(neighbor)) # ← debugging print
                    new_path = this_path + (neighbor,)
                    if goal_test(neighbor):
                        return new_path
                    agenda.append(new_path)
                    visited.add(neighbor)
        return None

    def solve_with_find_path(puzzle):
        """
        Returns a solution to the puzzle, or None if it cannot be solved.
        """

        def solution_to_state(solution):
            """Represent a solution as a tuple of sorted (key, value) dict item tuples."""
            return tuple(sorted(solution.items()))

        def state_to_solution(state):
            """Re-create a solution dict from its tuple representation."""
            return {domino: placement for (domino, placement) in state}

        def neighbors(state):
            solution = state_to_solution(state)
            # for each domino in lexicographical order
            for domino in sorted(puzzle["dominoes"]):
                # try to place it at coordinates in lexicographical order
                for (row, col) in sorted(puzzle["board"]):
                    for rotate in range(4):
                        neighbor = solution | {domino: (row, col, rotate)}
                        if solution_satisfies_constraints(puzzle, neighbor):
                            yield solution_to_state(neighbor)

        def goal(state):
            solution = state_to_solution(state)
            return (solution_covers_board(puzzle, solution) and
                    solution_satisfies_constraints(puzzle, solution))

        start = solution_to_state({})
        path = find_path(neighbors, start, goal, state_to_solution)
        return state_to_solution(path[-1]) if path else None
```